



COMPUTAÇÃO PARALELA

AULA 4





CONVERSA INICIAL

Nesta aula veremos os principais pontos para o desenvolvimento de softwares utilizando OpenMP e seu paradigma. O OpenMP é uma Interface de Programação de Aplicação, ou API (do inglês *Application Programming Interface*) desenvolvida para programação multiprocessada de memória compartilhada.

Objetivos da aula

Ao final desta aula esperamos atingir os seguintes objetivos, que serão avaliados ao longo da disciplina da forma indicada.

Tabela 1 – Objetivos e avaliações da aula 4

Objetivos	Avaliação
1 – Entendimento sobre os principais conceitos de OpenMP, principais comandos e suas finalidades	Atividade prática e questões dissertativas
2 – Desenvolvimento de programas simples OpenMP utilizando a ferramenta utilizando C++	Atividade prática e questões dissertativas
3 – Tratamento de problemas de concorrência em OpenMP	Atividade prática e questões dissertativas

TEMA 1 – INTRODUÇÃO AO OPENMP

Neste tema vamos discutir os primeiros conceitos sobre OpenMP. Tratando-se de programação multiprocessada, OpenMP é uma das principais APIs utilizadas hoje especialmente pela sua simplicidade e abstração em alto nível de algumas funcionalidades que demandam muita codificação adicional em outras APIs.



O OpenMP é compatível com as linguagens C, C++ e Fortran, foi desenvolvido em 1997 e no momento da escrita desta aula já está na versão 5.0 e continua sendo ativamente atualizada. Ele é gerido por um conselho sem fins lucrativos formado por um grupo de gigantes da produção de hardware e software, como Intel, AMD, IBM, Nvidia, entre outras. O foco da API está no desenvolvimento de aplicações para alto desempenho por meio do uso de paralelismo.

As soluções propostas pelo OpenMP são pensadas de maneira escalável, permitindo o desenvolvimento de softwares capazes de atuar em computadores comuns com pequeno número de núcleos como supercomputadores que atuam com milhares de núcleos.

Quando falamos de *clusters* pensamos em nós separados operando em conjunto conectados por uma rede. Como a memória não é unificada, dizemos que ele é um sistema de memória distribuída. Outra organização é responsável por uma solução chamada Open MPI (acrônimo em inglês para *Open Message Passing Interface*; em tradução livre: interface de passagem de mensagens), que é pensado para multiprocessamento em um ambiente de memória distribuída. Tanto o OpenMP quanto o Open MPI, embora tenham paradigmas diferentes, podem ser utilizados em conjunto a fim de formular soluções para aplicações que desejam desempenho máximo em um sistema como *cluster*. Open MPI e modelos híbridos que combinam as duas soluções serão discutido em maiores detalhes em outra aula.

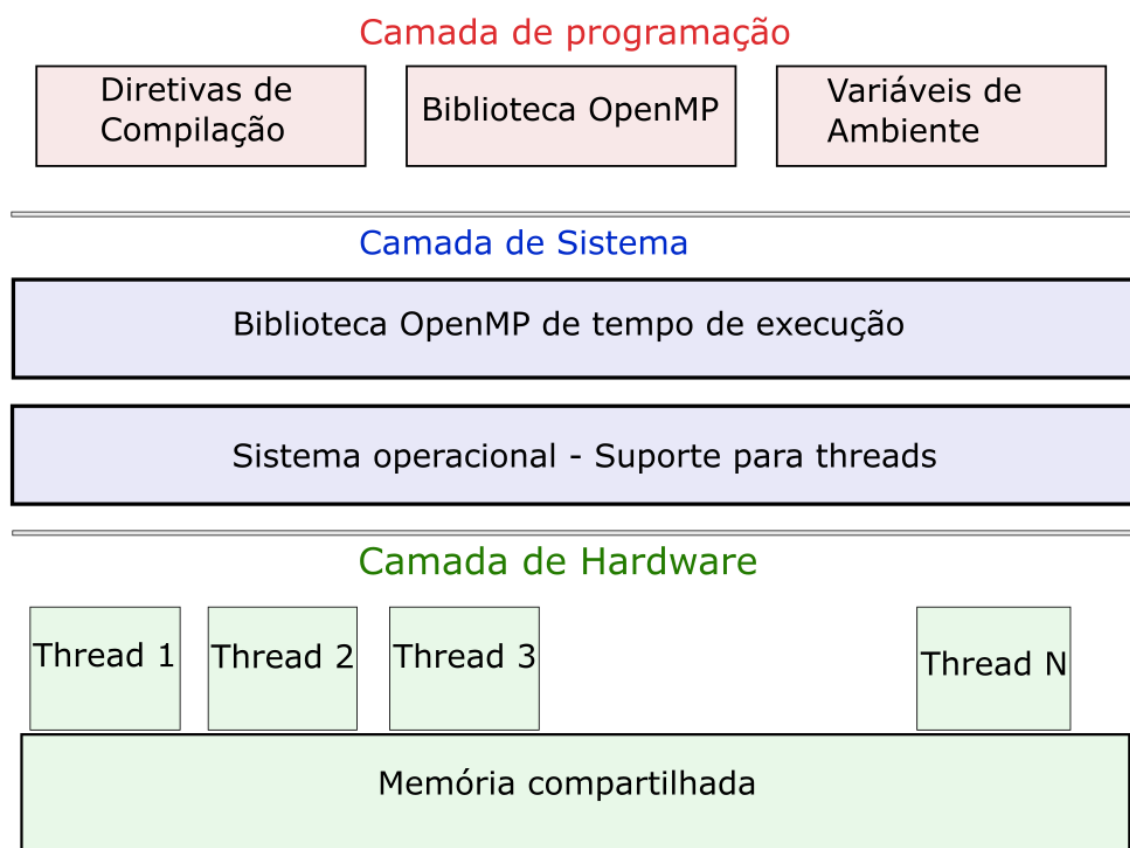
O maior trabalho do programador em paralelo está em descobrir as estratégias que melhor se encaixam em cada situação. Descobrir que trechos de código podem e quais não podem rodar em paralelo, e tanto quanto possível planejar estratégias paralelas para trechos originalmente não paralelizáveis. Também é tarefa do programador identificar os pontos críticos de sincronia do código e como minimizar eventuais perdas de desempenho e garantir que os problemas de sincronismo já discutidos não ocorram.

Existem iniciativas que buscam gerar compiladores que interpretam código escrito de maneira não paralela e tentam transformá-los automaticamente em uma versão paralela. No entanto, essas soluções ainda estão longe de apresentar um resultado satisfatório, e esse trabalho realmente cabe ao



programador. O OpenMP apresenta um conjunto de diretivas de compilação associadas a rotinas na biblioteca com foco em simplicidade dos comandos. Na Figura 1, vemos os principais componentes que relacionam o OpenMP divididos nas camadas de programação, que são responsabilidade do programador; a camada de sistema, que é responsabilidade do sistema operacional e dos desenvolvedores do OpenMP; e a camada de hardware, que depende dos equipamentos que compõem o processador.

Figura 1 – Representação nas camadas de programação, sistema e hardware do paralelismo *multithread*



TEMA 2 – PRIMEIRO PROGRAMA

Seguem algumas instruções e contextualizações para os nossos primeiros programas OpenMP, passando pela configuração e pela sintaxe.

2.1 Configuração

Os principais compiladores, dentre outras ferramentas, já contam com o OpenMP implementado, sem a necessidade de instalações adicionais. No



entanto, é necessário indicar ao compilador a necessidade de adotá-los. Configurações especiais podem ser necessárias em algumas IDEs como Visual Studio.

Para outros compiladores gcc amplamente adotado no linux e macOs. durante a compilação basta adicionar a opção `-fopenmp`:

```
= gcc -fopenmp codigo.c
```

Pronto! Com isso estamos preparados para seguir.

2.2 Código

A seguir, o código de um simples programa que imprime a mensagem “Adoro paralelismo” em múltiplas *threads*. Experimente executar em seu computador e vamos analisar em detalhes cada comando.

```
1 #include"omp.h"
2 #include"stdio.h"
3 using namespace std;
4
5 int main() {
6     printf("INICIO.\n");
7     #pragma omp parallel
8     {
9         printf("Adoro Paralelismo!\n");
10    }
11    printf("FIM.\n");
12    return 0;
13 }
```

Como resposta para esse código é esperado que seja exibida uma série de vezes a mensagem “Adoro Paralelismo” no seu console (entre duas e oito vezes, geralmente), cercados por uma única mensagem de INÍCIO e FIM. Se isso ocorreu no seu caso, parabéns, pois você realizou seu primeiro programa *multithreading*. Caso isso não ocorra, reveja os passos descritos na configuração. O número de vezes que a mensagem é exibida é a quantidade de *threads* que o OpenMP executou. Esse valor depende da quantidade de núcleos da máquina que se executa, mas é possível configurar uma quantidade específica de *threads* diferente para o OpenMP.

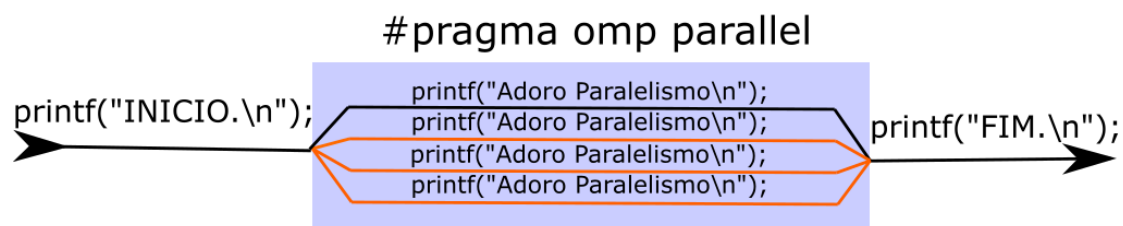


O comando **#include <omp.h>** (linha 1) inclui a biblioteca com as funções do OpenMP. Já o comando **#pragma omp parallel** (linha 7) é o responsável por indicar que o bloco de comando associado entre chaves deve ser executado em múltiplas *threads*. A quantidade exata de *threads* é definida pelo sistema da maneira que a escrevermos.

- O código **#pragma** indica uma diretiva ao compilador que deve tratar aquele trecho de código de alguma forma especial, no caso, paralelizando via OpenMP.
- Já o código **omp** indica que a diretiva será relacionada ao OpenMP.
- Enquanto o código **parallel** indica que o comando relacionado ao OpenMP é o paralelizar o código em anexo.

Observe que apenas o trecho entre chaves da diretiva **omp parallel** foi executado em múltiplas *threads*. As mensagens de início e fim foram apresentadas uma única vez. A ideia é justamente combinar trechos do código em que é conveniente paralelizar com trechos serializados para coordenar as execuções. A Figura 2 ilustra isso.

Figura 2 – Representação do código em execução a linha preta se divide em 4 *threads* e depois volta a ser uma única *thread*



TEMA 3 – COMANDOS BÁSICOS OPENMP

Neste tema vamos aprender os comandos essenciais que dão sentido ao OpenMP. Vamos analisar algumas das funções e diretivas básicas da biblioteca e suas finalidades.

3.1 `omp_get_thread_num()`



Para ter um pouco mais de controle sobre o que se está executando, é importante conseguir identificar individualmente cada *thread* e controlar a quantidade delas. Vamos analisar agora o código a seguir.

```
1 #include"omp.h"
2 #include"stdio.h"
3 int main() {
4     #pragma omp parallel num_threads(4)
5     {
6         int id = omp_get_thread_num();
7         printf("Sou a thread numero %d.\n", id);
8     }
9     return 0;
10 }
```

Observamos aqui dois comandos novos:

- **num_threads(4) (linha 4):** Esta diretiva é responsável por quantificar a quantidade de *threads* que será executada no bloco entre chaves.
- **omp_get_thread_num() (linha 6):** Nesta função temos a identificação individual de cada *thread*, no caso, 4 *threads*; cada uma receberá um valor entre 0 e 3.

Quanto ao comando **num_threads(4)**, associado ao `pragma omp parallel`, é importante observar que não é a única forma de modificarmos a quantidade de *threads* que o OpenMP gera. O comando **omp_set_num_threads(4)** também define o número de *threads* para 4, ou para o valor que desejar. Também é possível alterar a quantidade de *threads* modificando o valor da variável de ambiente `OMP_NUM_THREADS`. No Linux e no macOS, variáveis de ambiente são facilmente modificadas via comando **export** executados em um terminal. No Windows essa modificação geralmente é feita por meio da navegação das propriedades avançadas do sistema, dependendo da versão do Windows. Essa variável de ambiente por padrão é definida para o número de *threads* que o sistema nativamente suporta simultaneamente. Isso permite que um mesmo programa sem a necessidade de recompilação possa ser executado com diferente número de *threads*, dependendo apenas do valor da variável de ambiente.

Algo importante de observarmos que apesar das *threads* terem um valor de identificação individual como um número crescente, não existe nenhuma garantia quanto a ordem de execução. Caso o programa seja executado diversas



vezes, observaremos que a ordem das *threads* vai se alterar. Isso ocorre graças ao escalonamento de *threads* do sistema operacional que não oferece nenhuma garantia.

Para enfatizar o fato que não existe garantia na ordem que o código, adicione um segundo `printf` imediatamente abaixo do primeiro, com uma mensagem diferente, e veja o resultado ao executar.

```
...  
printf("Sou a thread numero %d.\n", id);  
printf("Segunda linha da thread %d.\n", id);  
...
```

Ao executar esse código, verificamos que as mensagens se misturam sem nenhuma ordem coerente e que se modificam em cada execução. É importante, então, termos em mente que ao paralelizarmos determinado trecho do código não deve fazer diferença ao resultado final a ordem das *threads*.

Existem várias formas de sincronizarmos *threads* em paralelo, como já discutimos na última aula, mutex e semáforos, por exemplo; no entanto, é importante fazer uma análise adequada das interações das *threads* com a memória e adotar esses mecanismos apenas em caso de última necessidade, pois, de maneira geral, sincronismo vem ao custo de desempenho, que é o objetivo principal do paralelismo.

3.2 `pragma omp parallel`

O OpenMP introduz um alto nível de abstração ao paralelismo, bibliotecas baixo nível, como *pthread*s, demandam comandos explícitos como o `pthread_create()` e `pthread_join()` que respectivamente criam e finalizam as *threads* por um *thread* mestre.

No OpenMP o mesmo acontece, porém de maneira transparente. A *thread* de índice 0 é a *thread* mestre que por debaixo dos panos usa comandos equivalentes ao `pthread_create()` o número de vezes adequado para a quantidade que se deseja e depois cuida de realizar o `pthread_join()` para encerrá-las. Uma *thread* é uma bifurcação da linha de código que vai executar de maneira concorrente; é possível que outra *thread* que não a mestre se bifurque novamente criando suas próprias *threads*, seguindo uma linha de



execução completamente diferente da mestre e usando a mesma diretiva **pragma omp parallel**. Isso pode ser repetido quantos níveis de profundidade for conveniente até o limite do sistema.

Observe o código a seguir.

```
1 int lista[1000];
2 #pragma omp parallel num_threads(4)
3 {
4     int id = omp_get_thread_num();
5     metodo(lista,id);
6 }
```

Temos um array chamado lista (linha 1) do tipo int declarada antes do **pragma omp parallel**, e temos uma variável id (linha 4) do tipo inteiro declarada dentro dela.

O array lista neste exemplo é compartilhado com todas as *threads*, as mudanças feitas por uma valem para todas. Já a variável id é considerada privada, ou seja, cada uma das quatro *threads* terá sua cópia individual de id.

É possível forçar uma variável declarada fora da área paralela a se tornar privada adicionando a diretiva **private(x,y)**, sendo x e y duas variáveis quaisquer, como exemplo a seguir.

```
1 int a=100,b=200,c=300;
2 #pragma omp parallel private(a,b,c)
3 {
4     c=metodo(a,b);
5 }
```

Neste caso cada *thread* terá uma cópia privada das variáveis **a,b** e **c**.

TEMA 4 – EXEMPLOS

Neste tema vamos estudar alguns problemas básicos, entender como pensar a programação em paralelo e avaliar o ganho que teremos com esses algoritmos.

4.1 Array quadrado

Suponha que desejamos criar um array em que o valor de cada posição seja igual ao seu índice elevado ao quadrado. Desta maneira: 1, 4, 9,16, 25, 36, ...



A forma natural de solucionar esse problema envolveria a criação de um *loop* que passe por todas as posições fazendo o cálculo. Para paralelizar isso podemos distribuir parte dos índices entre cada *thread*. Se tivermos oito *threads*, cada uma ficaria com um oitavo do trabalho; a primeira *thread* com o primeiro oitavo, a próxima com o segundo oitavo, e assim por diante. Então, temos que encontrar um índice de início e fim para cada *thread* baseado no seu *id*, lembrando que o *id* é a única informação que distingue cada *thread*. Se desejamos criar um código que funcione para qualquer número de *threads*, podemos pensar em um algoritmo como o descrito a seguir – pegamos o total de elementos dividido pelo número de *threads* para calcular em quantos elementos cada *thread* vai atuar. E multiplicamos esse valor pelo *id* da *thread* para identificar qual será esse intervalo. Confira como fica essa abordagem no código a seguir.

```
1 #include <stdio.h>
2 #include <omp.h>
3 #define TOTAL 2048
4
5 int main() {
6     int A[TOTAL];
7     #pragma omp parallel
8     {
9         int id = omp_get_thread_num();
10        int nt = omp_get_num_threads();
11        int tam = TOTAL / nt;
12        int ini = id * tam;
13        int fim = ini + tam - 1;
14        int i;
15        for (int i = ini; i < fim; i++){
16            A[i] = i*i;
17            printf("Th[%d] : %02d = %03d\n", id, i, A[i]);
18        }
19    }
20    return 0;
21 }
```

Conforme dito no parágrafo anterior, definimos o início (linha 12) e o fim (linha 13) do intervalo de índices no qual cada *thread* trabalhará baseado no número de *threads* (linha 10) e na *id* (linha 9) de cada uma.

Essa estratégia funciona e é aplicada variações bem próximas dela em diversas linguagens paralelas. No entanto, como mencionado, o OpenMP possui



um elevado nível de abstração e tem uma alternativa para essa estrutura de código que é muito comum.

Sem pensar em programação paralela, o algoritmo mais simples que resolve esse problema consiste em um *loop* que passe por todas as posições do *array* e simplesmente multiplique o índice por ele mesmo e armazene na sua respectiva posição.

Para facilitar o trabalho da divisão da tarefa em um *loop* e rapidamente desenvolver códigos como esse, o OpenMP conta com uma diretiva especial que simplifica bastante esta atividade **#pragma omp parallel for**. Confira o código a seguir. Basta a adição da diretiva ao código.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4 #define TOTAL 2048
5 int main() {
6     int A[TOTAL];
7     #pragma omp parallel for
8     for (int i = 0; i < TOTAL; ++i) {
9         A[i] = i*i;
10        printf("Th[%d]: %02d=%03d\n", omp_get_thread_num(), i, A[i]);
11    }
12    system("pause");
13    return 0;
14 }
```

Os dois códigos em essência fazem a mesma coisa, distribuem entre uma quantidade de *threads* dada pelo sistema as iterações do loop for, cada *thread* operando com um conjunto de índices diferentes. Vemos que a segunda solução, dada pela diretiva **#pragma omp parallel for** (linha 7) simplifica bastante o código necessário para esta atividade que é muito recorrente em programação paralela.

É importante observar que essa diretiva automaticamente divide e mapeia parte dos índices do *loop* para cada *thread*. Ela não garante uma ordem de execução e nenhum tipo de tratamento de sincronismo.

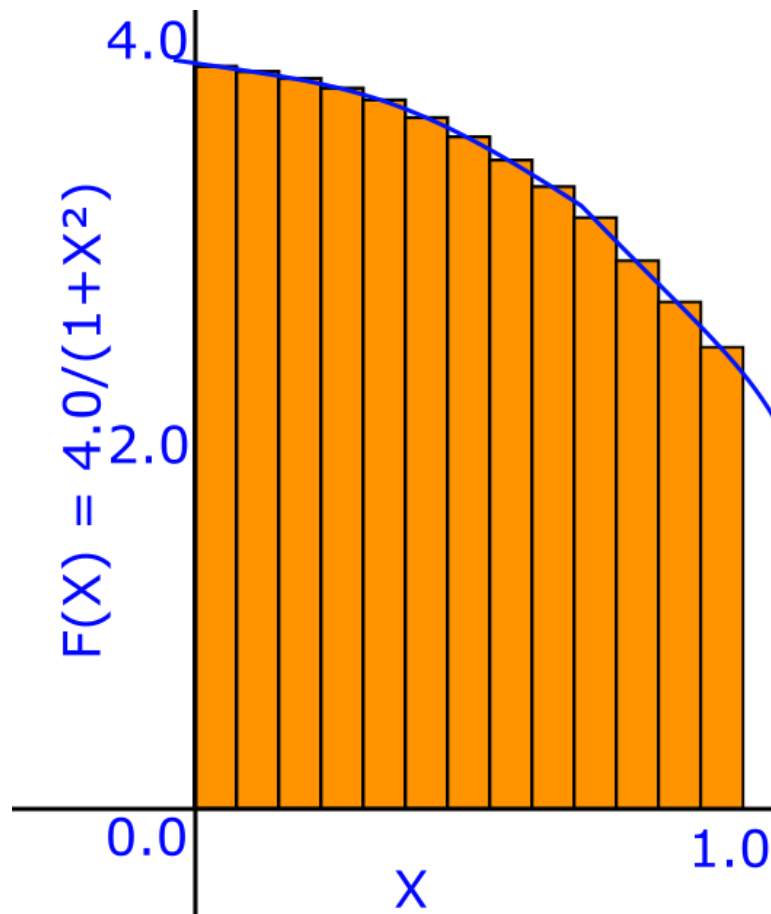
4.2 Cálculo do valor de π

Outro exemplo interessante de analisarmos é o cálculo de integral. Mais especificamente $\int_0^1 \frac{4.0}{(1+x^2)}$, que lê-se como sendo a integral no intervalo entre



0 e 1 da função $f(x) = 4 / (1 + x^2)$. Lembrando que a integral graficamente se traduz como a área abaixo de uma função. Veja a Figura 3 para compreender melhor e visualmente o que desejamos calcular. O interessante dessa integral é que sua resposta é exatamente o valor de π (PI), o que nos permite facilmente validar a precisão das respostas obtidas.

Figura 3 – Integral calculada em blocos da função $f(x) = 4 / (1-x^2)$



A estratégia básica consiste em gerar diversos retângulos a cada pequeno intervalo fixo no eixo X e calculamos a integral pela soma desses vários retângulos. O algoritmo que realiza esse cálculo é bastante simples. Em linha gerais, precisamos saber a altura e a largura de cada retângulo para calcular suas áreas e somá-los; para isso, seguimos os seguintes passos visualizado no código a seguir:

- Definimos uma quantidade de retângulos (linha 1).
- A largura de cada retângulo, chamaremos aqui de L , é medida pela largura total dividida pela quantidade de retângulos (linha 5).



- Para a altura de cada retângulo, primeiro calculamos o ponto central do retângulo no eixo X e multiplicamos pelo resultado da função $f(x) = 4 / (1 - x^2)$ (linha 9).
- O ponto central de cada retângulo no eixo X será o número do retângulo multiplicado por $L + L/2$.
- Somamos todas as alturas de todos os retângulos (linha 11).
- Multiplicamos a soma das alturas pela largura (linha 14).

Abaixo o código que soluciona esse problema de forma não paralela.

```
1 #define RETANGULOS 100000
2 int main() {
3     double x, pi, soma = 0;
4     //largura de cada retângulo
5     largura = 1.0 / RETANGULOS;
6
7     for (int i = 0; i < RETANGULOS; i++) {
8         //cálculo do ponto central
9         x = (i + 0.5) * largura;
10        //cálculo da altura do ponto central
11        soma += 4 / (1 + x * x);
12    }
13    //multiplicação das alturas pela largura
14    pi = largura * soma;
15 }
```

Esse é um exemplo clássico para o paralelismo, pois podemos facilmente verificar o nível de precisão da resposta comparando a quantidade de casas decimais iguais ao valor de π e ele tem um ganho de desempenho cada vez maior com o número crescente de *threads* trabalhando em paralelo.

Baseado no exercício anterior, a primeira ideia que talvez lhe ocorra seja de simplesmente utilizar a diretiva **#pragma omp parallel for**, no entanto, essa abordagem não funciona nessa situação, pois como todas as *threads* estão acessando a mesma variável soma, ocorre uma condição de corrida sobre essa variável, gerando resultados inconsistentes.

A solução clássica para esse problema consiste em permitir que cada *thread* faça o seu somatório individual. Digamos que tenhamos três *threads*. Então, a primeira somará o primeiro um terço, a próxima *thread* somará o segundo terço, e a última *thread* somará o último um terço. E, ao final, juntamos



os três somatórios. Temos o mesmo análogo para qualquer outra quantidade de *threads*. Veja a seguir o código que implementa essa solução.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define NUM_THREADS 8 //PARALELA
6 #define RETANGULOS 100000000
7
8 int main()
9 {
10     double largura;
11     int i, nthreads;
12     //array para armazenar as somas
13     double pi, soma[NUM_THREADS];
14     largura = 1.0 / (double)RETANGULOS;
15     //calcula tempo
16     double tempo_inicio = omp_get_wtime();
17
18     omp_set_num_threads(NUM_THREADS);
19     #pragma omp parallel
20     {
21         int i, id, nthrds;
22         double x;
23         //Recupera o numero da thread
24         id = omp_get_thread_num();
25         //verdadeiro num de threads
26         nthrds = omp_get_num_threads();
27         if (id == 0)
28             nthreads = nthrds;
29         //distribuição cíclica (distribuição round robin)
30         for (i = id, soma[id] = 0.0; i < RETANGULOS; i = i + nthrds)
31         { //id = 0, temos 0, 4, 8, 12... id = 1, temos 1, 5, 9, 13...
32             x = (i + 0.5) * largura;
33             soma[id] = soma[id] + 4.0 / (1.0 + x * x);
34         }
35     }
36     for (i = 0, pi = 0.0; i < nthreads; i++)
37         pi = pi + soma[i] * largura;
38     //calcula tempo
39     double tempo_fim = omp_get_wtime();
40     printf("%f\n", pi);
41     printf("%f s\n", (tempo_fim - tempo_inicio));
42     system("pause");
43     return 0;
44 }
```

Nesse código vemos mais um comando do OpenMP o **omp_get_wtime()** (linhas 16 e 39), que mensura quantidade de tempo passado em segundos. Ao medirmos antes e depois da execução paralela e tirarmos a diferença dos valores, temos o tempo gasto naquele trecho de código. Assim, conseguimos facilmente mensurar o ganho de tempo com o aumento no número de *threads*.



Sobre o número de *threads* é importante observar outra situação. É possível solicitar quantas *threads* se deseja pelo comando **omp_set_num_threads()** (linha 18); no entanto, não temos garantia de que o valor solicitado será o valor entregue realmente, o real valor entregue depende de configurações dentro do sistema operacional que pode optar por entregar uma quantidade menor. Por essa razão, dentro do código executamos a linha de código: **nthrds = omp_get_num_threads()** (linha 26); que retorna o real número de entregue de *threads*.

O algoritmo apresentado trabalha com a ideia de dividir o trabalho de soma entre as *threads* em uma distribuição cíclica. Suponha que existam quatro *threads*; a *thread* 0 trabalha nos índices múltiplos de 4, a *thread* 1 trabalha nos índices múltiplos de 4 + 1, a *thread* 2 trabalha nos múltiplos de 4 + 2, e a *thread* 3 trabalha nos múltiplos de 4 + 3 (linha 30).

A implementação apresentada tem uma limitação de ordem estrutural. As diferentes *threads* acessam o mesmo array **soma** (linha 33) para registrar o somatório de cada *thread*. Como tratam-se de operações de escrita por diferentes *threads* executando em diferentes núcleos de processamento, acontece um fenômeno chamado de compartilhamento falso. As posições dentro do array são vizinhas, portanto, estão no mesmo bloco de memória do ponto de vista de mapeamento de cache, e o mesmo bloco de memória sendo atualizado diversas vezes por núcleos de processamento, cada uma com seu próprio *cache* acarreta a necessidade de que todas as atualizações sejam feitas na memória principal para garantir a coerência entre os *caches*, tornando o processamento significativamente mais custoso.

Essa questão é muito específica de cada processador, mas é o tipo de situação interessante de se analisar para se entender otimização baixo nível. Voltaremos a esse problema no próximo tema, com uma solução que leva esse aspecto em consideração.

TEMA 5 – SINCRONISMO

Neste tema discutiremos os principais comandos de sincronismo e suas aplicações utilizando OpenMP.



O tópico de sincronismo é muito vasto na literatura acadêmica, e aqui abordaremos alguns dos principais usos dessa tecnologia.

5.1 Barrier

Esta forma de sincronismo simplesmente faz com que todas as *threads* pausem suas execuções nessa barreira virtual até que todas as *threads* cheguem na barreira. Acompanhe o código a seguir, que apresenta um exemplo de uso desse mecanismo.

```
1 #pragma omp parallel
2 {
3     int id = omp_get_thread_num();
4     A[id] = funcao_complexa_1(id);
5
6     #pragma omp barrier
7     B[id] = funcao_complexa_2(A, id);
8 }
```

Nesse exemplo temos duas chamadas de funções hipotéticas, uma **funcao_complexa_1** (linha 4), que vai preencher um determinado *array A*. Suponha agora que esse *array* se faça necessário para a execução de uma **funcao_complexa_2** (linha 7). Como o código está em um trecho paralelo, é possível que uma *thread* inicie a segunda função, enquanto outras ainda estão na primeira, ou seja, o *array A* estará incompleto e o cálculo que depende dele já estará em andamento, gerando resultados indesejados. Para resolver isso o uso do **#pragma omp barrier** (linha 6) soluciona esse problema de sincronismo.

5.2 Mutual exclusion

No OpenMP é muito simples de implementar *mutual exclusion*; basta utilizar a diretiva **#pragma omp critical** no trecho no qual desejamos que seja executado por uma única *thread* por vez. Veja o código a seguir, em que exemplificando essa situação.

```
1 #pragma omp parallel
2 {
3     float B; int i, id, nthreads;
4     id = omp_get_thread_num();
5     nthreads = omp_get_num_threads();
```




```
6      for (int i = id; i < fim; i += nthreads) {
7          B = funcao_complexa_1(i);
8      }
9      #pragma omp critical
10     res += funcao_complexa_2(B);
11 }
```

Nesse exemplo, o código **res += funcao_complexa_2(B);** (linha 10) será executado por uma *thread* de cada vez. A diretiva **critical** (linha 9) também permite o uso de chaves para ser associada a um bloco de código ao invés de uma linha única. No entanto, é importante salientar mais uma vez que as funções de sincronismo devem ser utilizadas com muito critério. Preferencialmente usá-las nos menores trechos possíveis, pois perde-se desempenho e corre-se o risco de serializar o código.

A diretiva **atomic** segue o mesmo princípio da diretiva **critical** no entanto ela tenta se aproveitar de certas propriedades dos sistemas nos quais executam. Os processadores hoje são projetados para otimizar determinadas funcionalidades simples que geram problemas de sincronismo, digamos uma simples, como na tabela abaixo.

Tabela 2 – Exemplo de tabela com problemas de sincronismo

x operador= expressão; (ex: x+= 3; x -= y*y;)
x++;
++x;
x--;
--x;

A diretiva **critical** pode sempre substituir **atomic**, no entanto, **atomic** apresentará um ganho no desempenho por se aproveitar de otimizações nos processadores para comandos, como especificados na tabela.



5.3 Cálculo do valor de PI sincronizado

Agora que conhecemos o comando de sincronismo, é possível escrever uma versão um pouco mais simples de algoritmo de cálculo de PI pela integral. Confira a seguir.

```
1  #include <omp.h>
2  #define NUM_THREADS 8 //PARALELA
3  #define RETANGULOS 100000000
4  int main(){
5      double largura;
6      int i, nthreads;
7      double pi, soma;
8
9      largura = 1.0 / (double)RETANGULOS;
10     double tempo_inicio = omp_get_wtime();
11     omp_set_num_threads(NUM_THREADS);
12     #pragma omp parallel
13     {
14         int i, id, nthrds; double x;
15         //numero identificador da thread
16         id = omp_get_thread_num();
17         //verdadeiro num de threads
18         nthrds = omp_get_num_threads();
19         if (id == 0)
20             nthreads = nthrds;
21         for (i= id, soma= 0; i < RETANGULOS; i+= nthrds){
22             x = (i + 0.5) * largura;
23             soma = soma + 4.0 / (1.0 + x * x);
24         }
25         soma = soma*largura;
26         #pragma omp atomic
27         pi+= soma;
28     }
29     //calcula tempo
30     double tempo_fim = omp_get_wtime();
31     printf("%f\n", pi);
32     printf("%f s\n", (tempo_fim - tempo_inicio));
33     system("pause");
34     return 0;
35 }
```

No código apresentado, além de mais simples que o anterior, pois não precisamos mais de uma *array* de respostas parciais, também temos um ganho no desempenho pelo fato de a diretiva **atomic** (linha 26) ser mais apropriada para o cenário proposto sem criar um problema de falso compartilhamento da cache.



FINALIZANDO

Nesta aula, realizamos atividades práticas de programação utilizando OpenMP. Aprendemos suas diretivas e vimos exemplos com paralelismo e sincronismo.